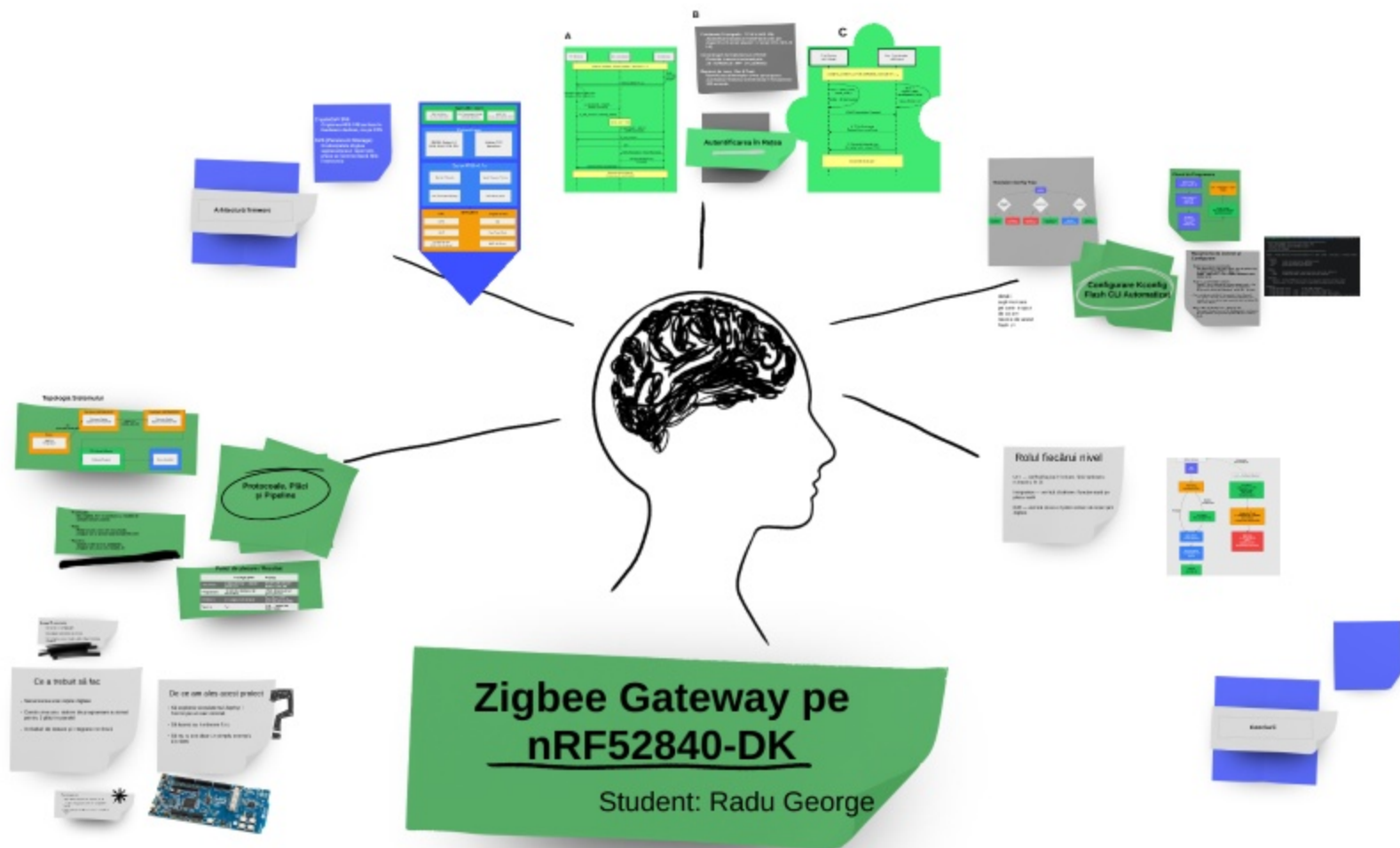




Zigbee Gateway pe nRF52840-DK

Student: Radu George

Scopul Proiectului:

- Un senzor de temperatură
- Două plăci care comunică wireless
- Un computer care primește datele integrat cu home assistant

Ce a trebuit să fac

Realizarea unei rețele Zigbee

Realizarea unui sistem de programare automat
al 2 plăci în paralel

Testări de testare și integrare continuă

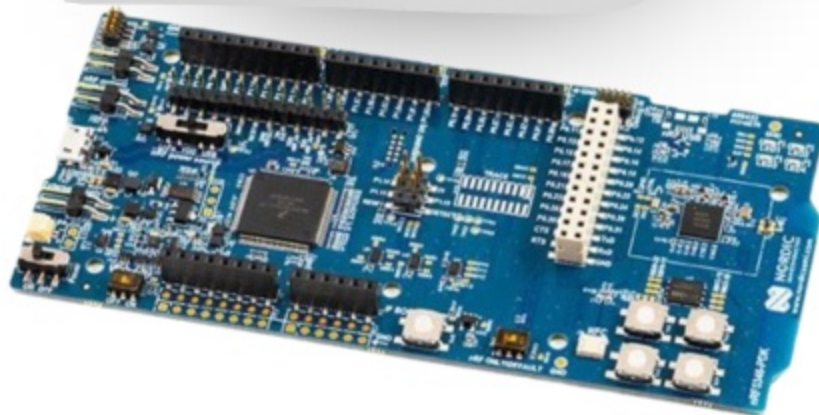
De ce am ales acest proiect

- Să explorez ecosistemul Zephyr / Nordic pe un caz concret
- Să lucrez cu hardware fizic
- Să nu ruleze doar un simplu exemplu din SDK



Ce am rezolvat

- Rețea Zigbee, funcțională pe hardware fizic real
- Un singur script ajutător: compilare + programare + validare
- Testare pe etape, până la comunicație completă între 2 plăci



Ce a trebuit să fac

- Securizarea unei rețele Zigbee
- Construirea unui sistem de programare automat pentru 2 plăci în paralel
- 3 niveluri de testare și integrare continuă

De c

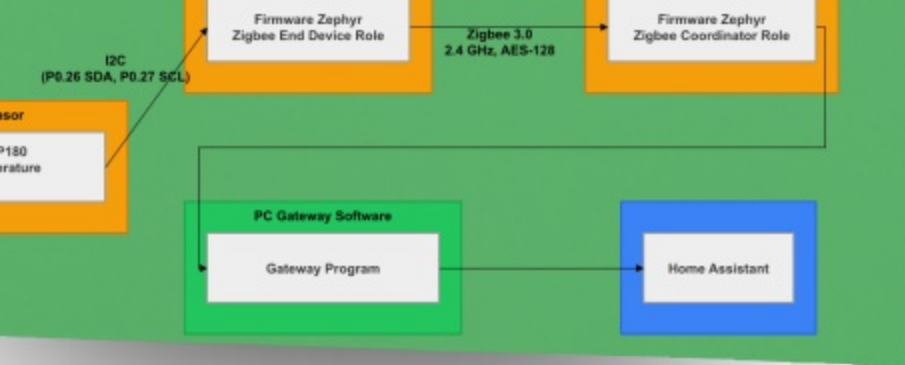
- Să exp
Nordic
- Să luc
- Să nu
din SD



Ce am rezolvat

- Rețea Zigbee, funcțională pe hardware fizic real
- Un singur script ajutător: compilare + programare + validare
- Testare pe etape, până la comunicație completă între 2 plăci





Protocoale, Plăci și Pipeline

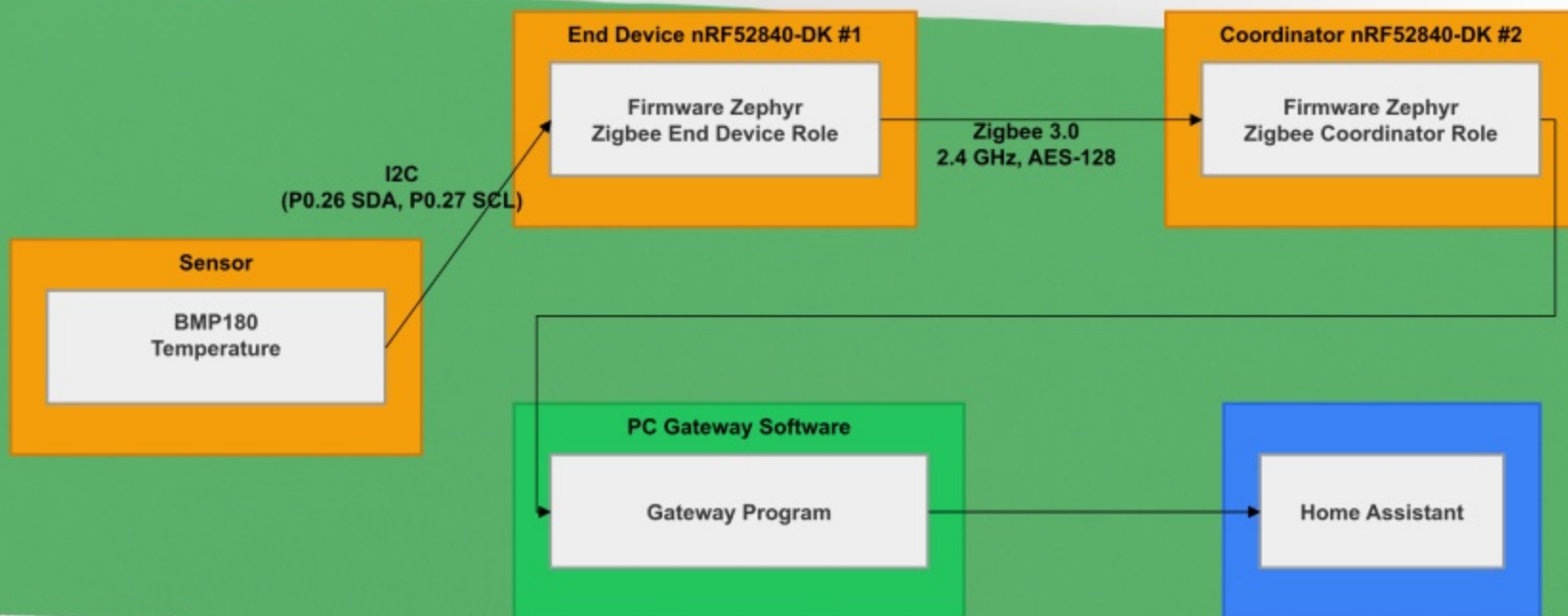
- **Protocoale:**
I2C, Zigbee, Home assistant au modele de execuție incompatibile
- **Plăci:**
Roluri opuse, asociere securizată, programare și testare coordonată între ele
- **Pipeline:**
Construit de la zero: compilare, programare, asociere, testare, CI

Punct de plecare / Rezultat

	Exemple SDK	Proiect
Securitate	Global Link Key — oricine poate intra	Install Code unic per device + AES-128
Programare	~4 comenzi manuale, căi hardcodeate	.Jflash_devices.sh all --prod --persist
Firmware	Un singur profil de build	Dev / Prod / E2E — controlat prin Kconfig

- **Protocoloale:**
I2C, Zigbee, Home assistant au modele de execuție incompatibile
- **Plăci:**
Roluri opuse, asociere securizată, programare și testare coordonată între ele
- **Pipeline:**
Construit de la zero: compilare, programare, asociere, testare, CI

Topologia Sistemului



Punct de plecare / Rezultat

	Exemple SDK	Proiect
Securitate	Global Link Key — oricine poate intra	Install Code unic per device + AES-128
Programare	~4 comenzi manuale, căi hardcodate	<code>./flash_devices.sh all --prod --persist</code>
Firmware	Un singur profil de build	Dev / Prod / E2E — controlat prin Kconfig
Testare	Zero	Unit → Integration → E2E + CI/CD

Arhitectură firmware

CryptoCell-310:

Criptarea AES-128 se face în hardware dedicat, nu pe CPU

NVS (Persistent Storage):

Credențialele Zigbee supraviețuiesc repornirii, placa se reconectează fără intervenție

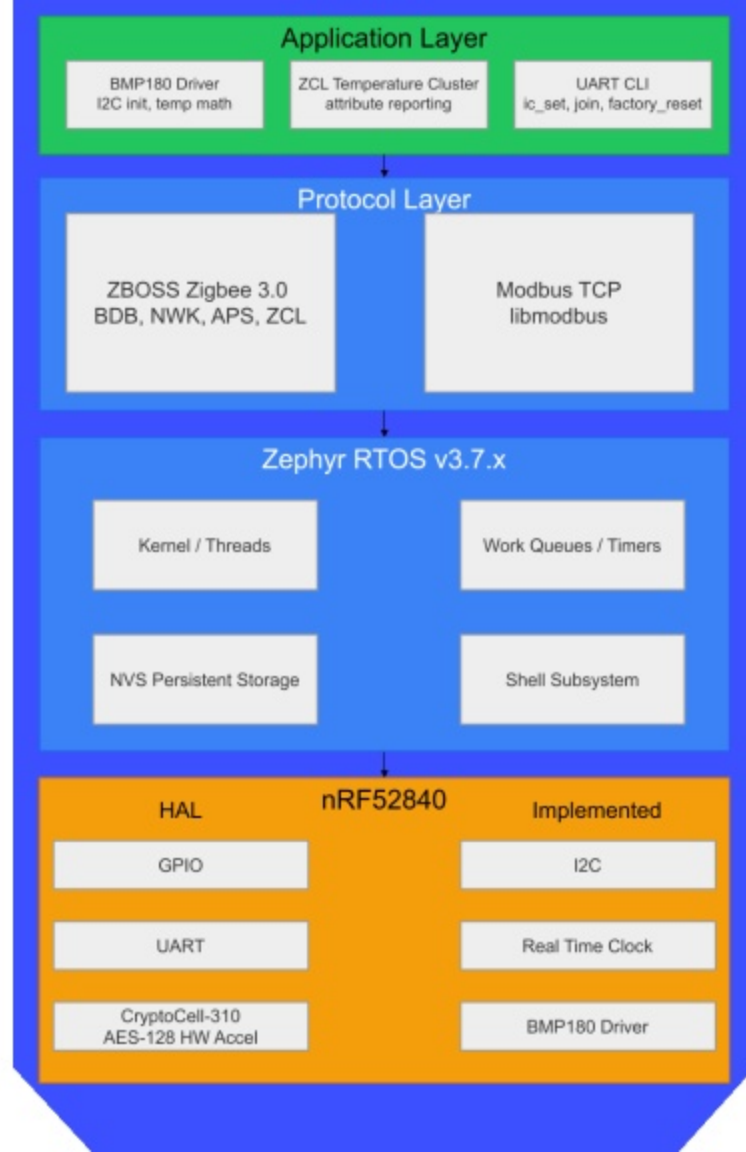


CryptoCell-310:

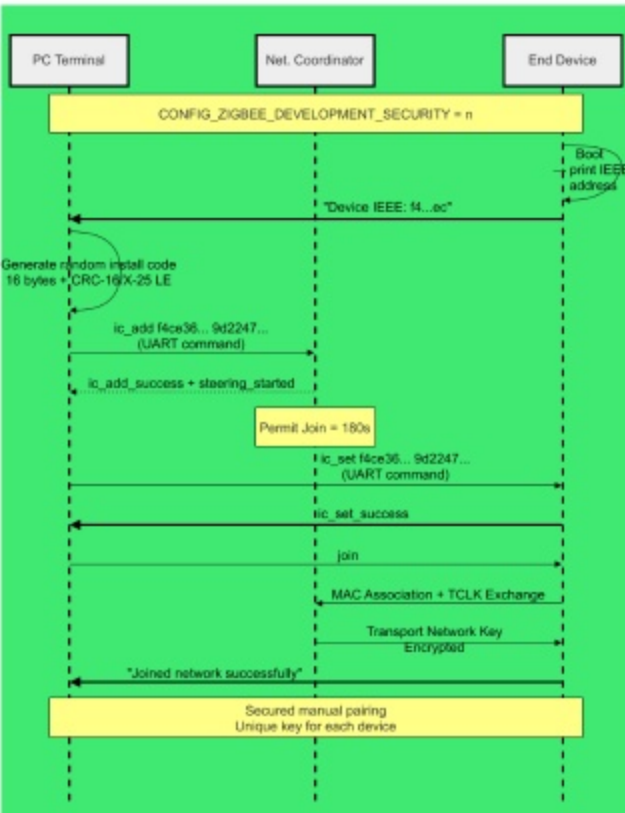
Criptarea AES-128 se face în hardware dedicat, nu pe CPU

NVS (Persistent Storage):

Credențialele Zigbee supraviețuiesc repornirii, placa se reconectează fără intervenție



A

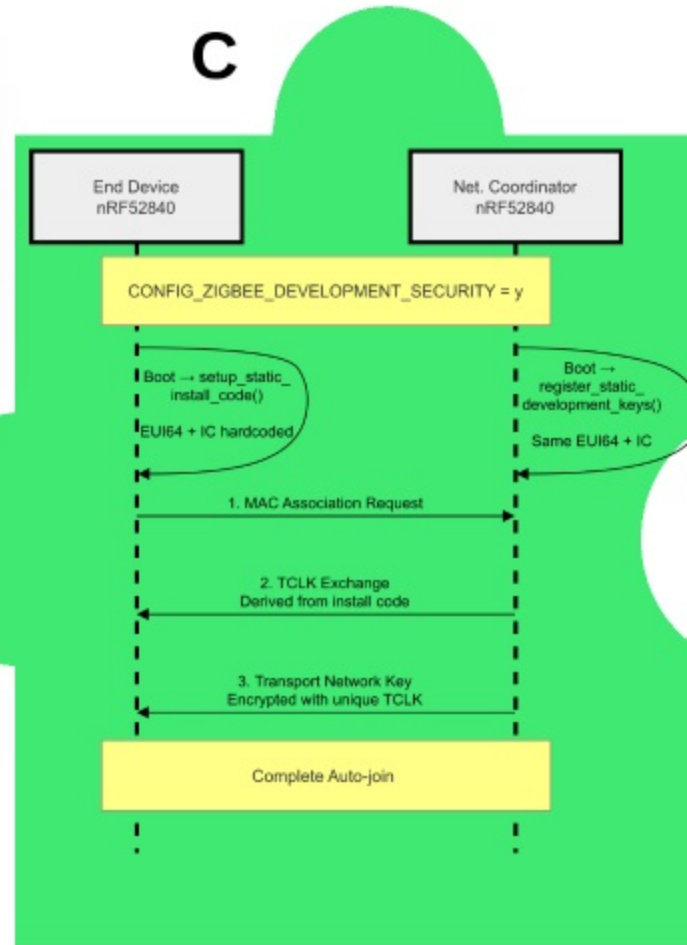


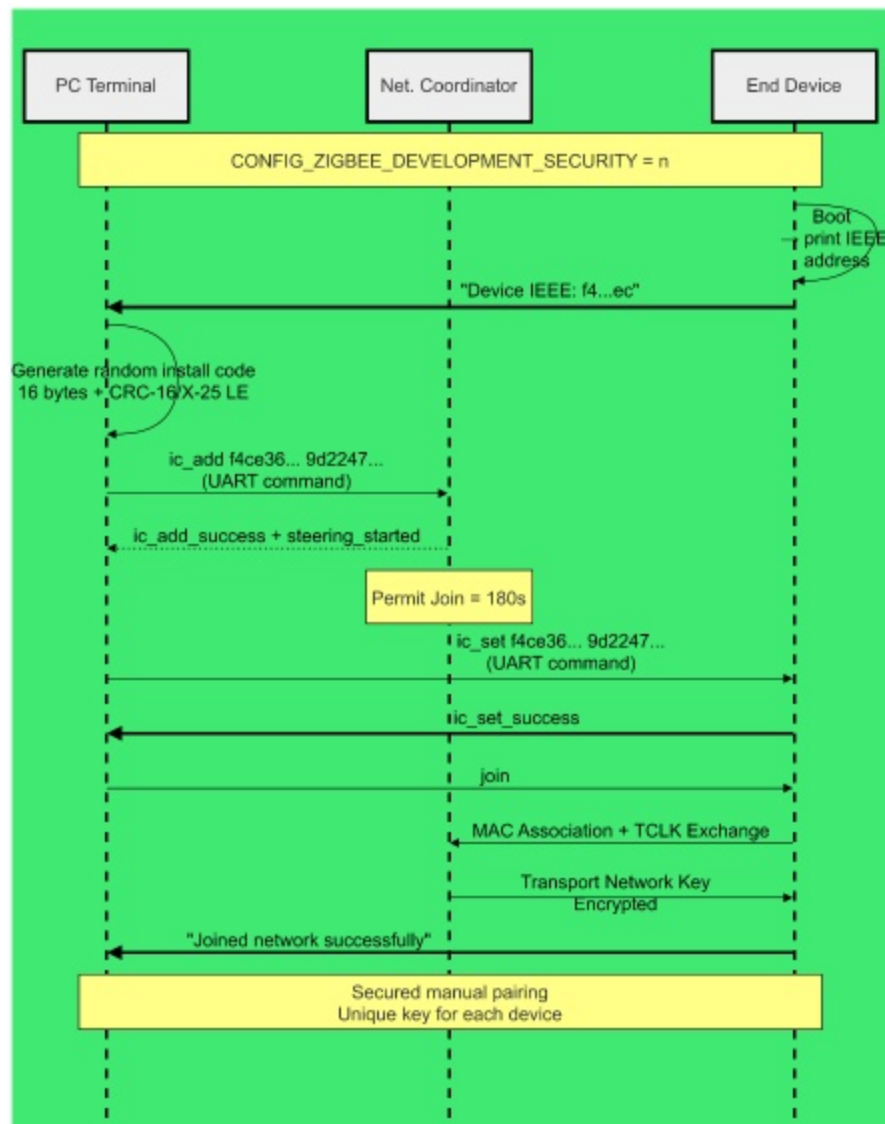
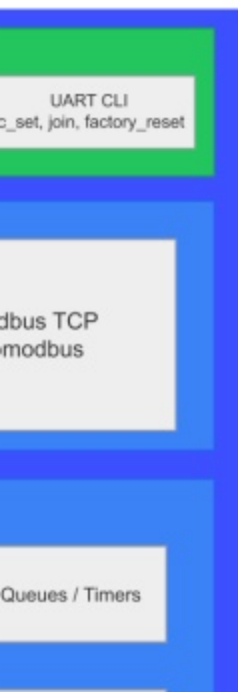
B

- Fundament Criptografic - TCLK & AES-128:
Autentificare bazată pe Install Code unic per dispozitiv (16 octeți aleatori + 2 octeți CRC-16/X-25 LE).
- Constrângeri de Implementare ZBOSS:
Protecția la acces concurrent prin ZB_SCHEDULE_APP_CALLBACK()
- Regimuri de lucru - Dev & Prod:
Identificarea diferențelor dintre parcurgerea automată și înrolarea restricționată în fereastră de 180 secunde.

Autentificarea în Rețea

C





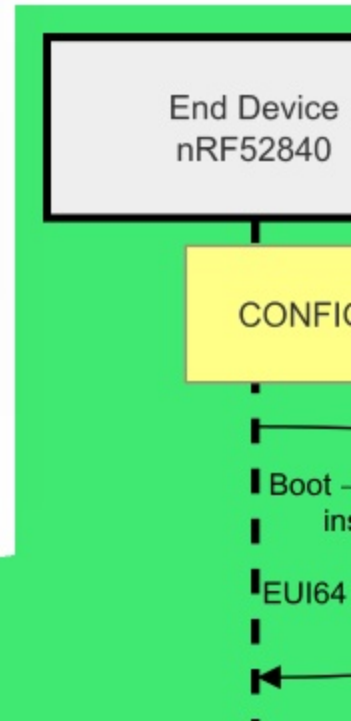
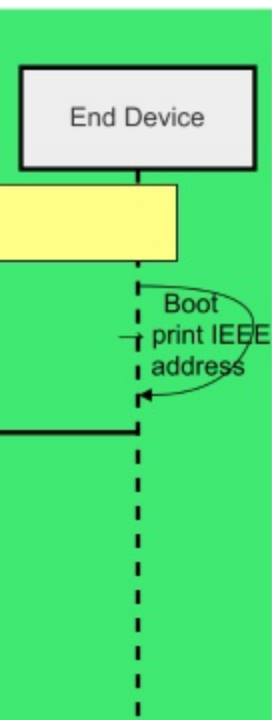
dispozitiv (16 octeți aleator
LE).

- Constrângeri de Implementare
Protecția la acces concuren
ZB_SCHEDULE_APP_CALL
- Regimuri de lucru - Dev & Pro
Identificarea diferențelor di
automată și înrolarea restric
180 secunde.

Autentificarea

B

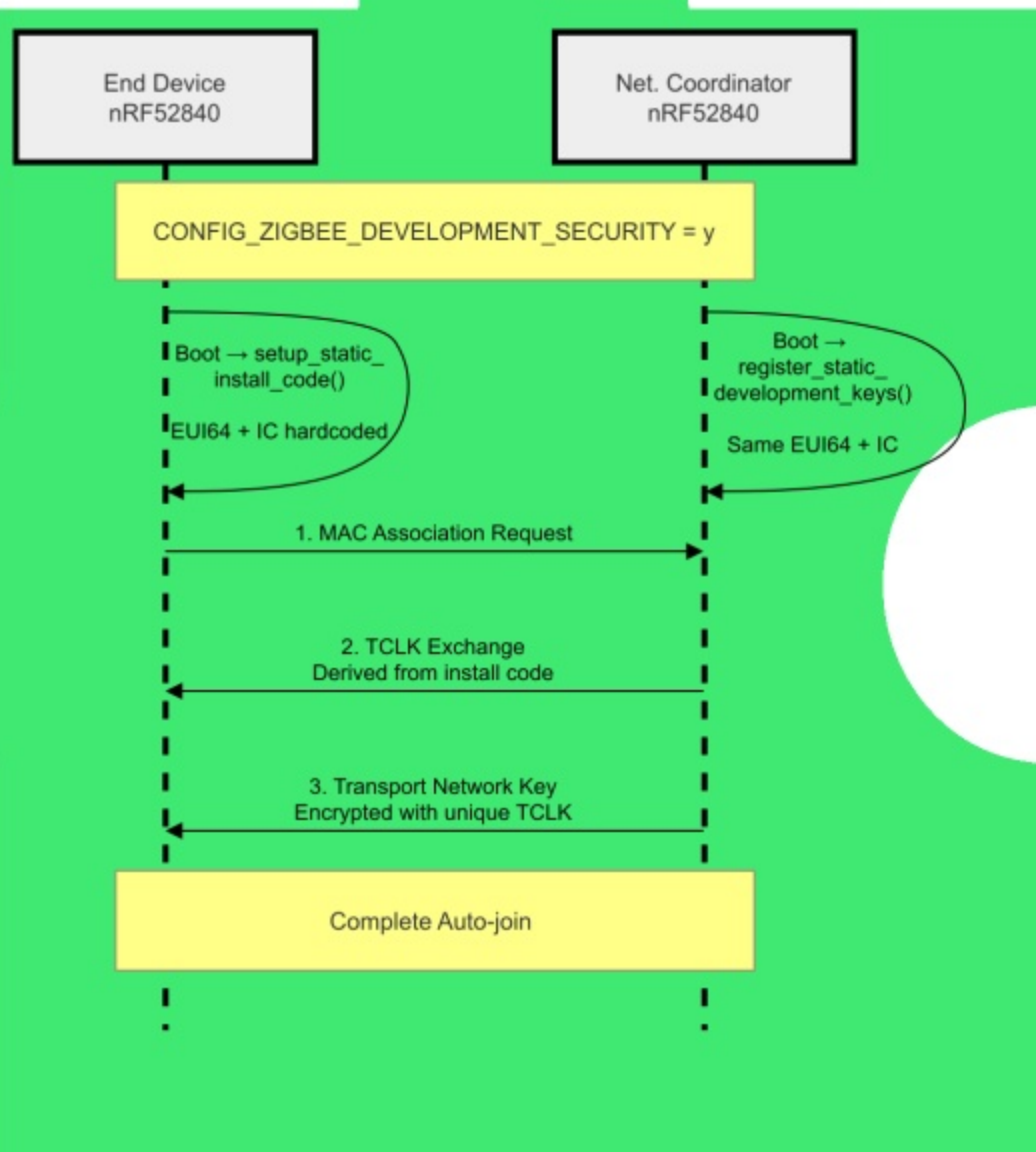
- Fundament Criptografic - TCLK & AES-128:
Autentificare bazată pe Install Code unic per dispozitiv (16 octeți aleatori + 2 octeți CRC-16/X-25 LE).
- Constrângeri de Implementare ZBOSS:
Protecția la acces concurent prin
`ZB_SCHEDULE_APP_CALLBACK()`
- Regimuri de lucru - Dev & Prod:
Identificarea diferențelor dintre parcurgerea automată și înrolarea restricționată în fereastră de 180 secunde.



mentare ZBOSS:
concurrent prin
PP_CALLBACK()

ev & Prod:
nțelor dintre parcurgerea
ea restricționată în fereastră de

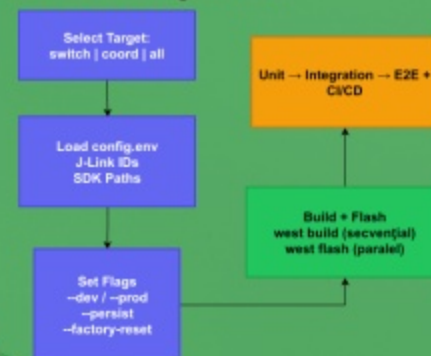
area în Rețea



Decision Config Tree



Fluxul de Programare



Configurare Kconfig Flash CLI Automatizat

detalii
suplimentare
pe care le spun
de ce am
nevoie de acest
flash cli

Mecanisme de Control și Configurare

- Control la Compilare prin Kconfig:
Comutare între 3 profile (Dev, Prod, Test) folosind 3 flag-uri custom (DEVELOPMENT, SECURITY, RESET_PERSISTENT, TEST_MODE), fără modificarea codului sursă.
- Mapare CLI și Protecție la Eroare:
Scriptul `./flash_devices.sh` traduce argumentele CLI în opțiuni Kconfig și execută verificări de siguranță (avertisment automat la apelarea `--prod` fără `--persist`).
- Execuție Optimizată (Build Secvențial + Flash Paralel):
Compilare ordonată pentru stabilitatea mediului west, urmată de scriere simultană pe ambele plăci utilizând ID-urile unice J-Link.
- Depozit Unic de Configurare și Stare Curată:
Decuplare totală a parametrilor hardware prin `config.env` și suport pentru ștergerea integrală NVM prin `--factory-reset`.

```

george@george-KLUL-W039
=====
Mode: DEVELOPMENT (auto)
Storage: VOLATILE (reset)
Flash action: NORMAL
=====

Usage: ./flash_devices.sh [-h] [-v] [-d] [-p] [-t] [-s] [-f] [-r] [-e]

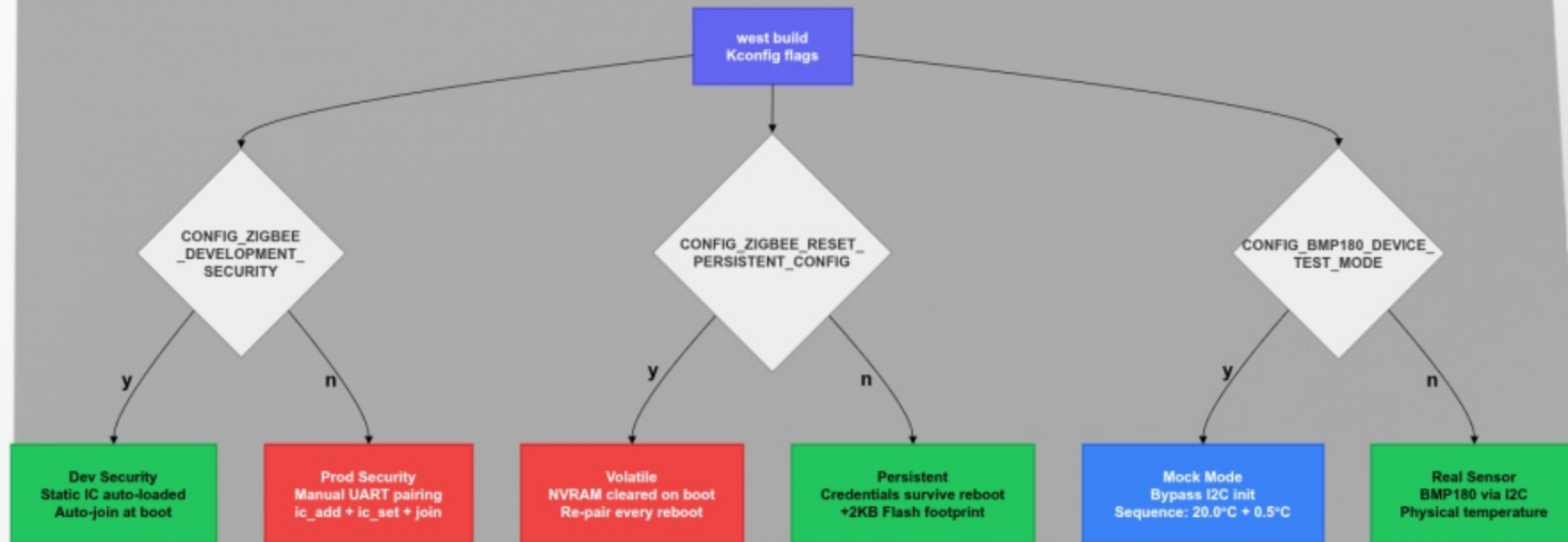
Targets:
  switch  Flash the switch
  coord   Flash the coordinator
  all      Flash all targets

Modes:
  --dev    Development mode
  --prod   Production mode

Options:
  --persist  Enable persistent storage
  --factory-reset  Factory reset the device

Examples:
  ./flash_devices.sh all
  ./flash_devices.sh switch
  ./flash_devices.sh coord
  ./flash_devices.sh all --persist
  ./flash_devices.sh all --factory-reset
  
```

Decision Config Tree



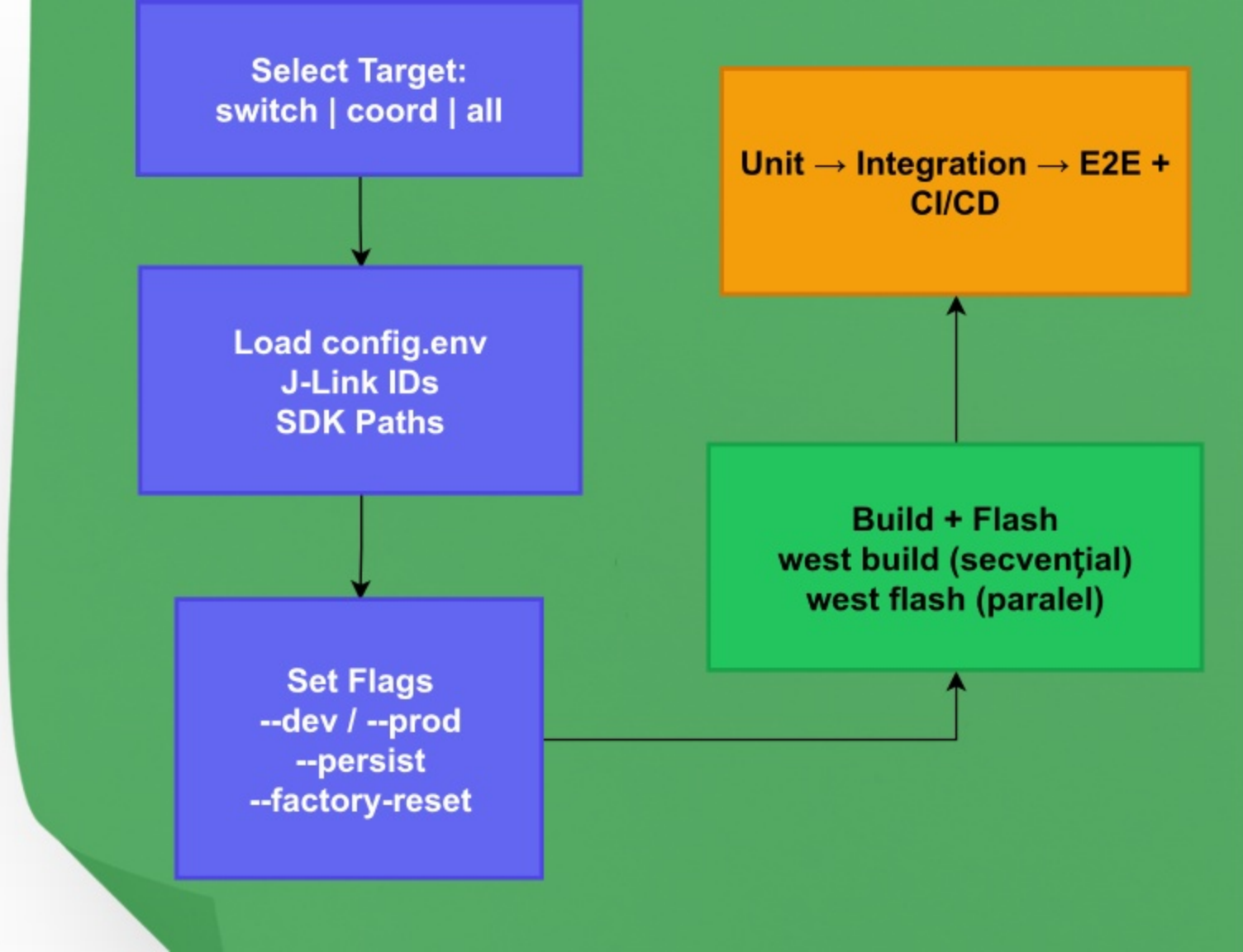
Select Target:
switch | coord | all

Load config.env
J-Link IDs
SDK Paths

Set Flags
--dev / --prod
--persist
--factory-reset

**Unit → Integration → E2E +
CI/CD**

Build + Flash
west build (secvențial)
west flash (paralel)



Mecanisme de Control și Configurare

- Control la Compilare prin Kconfig:
Comutare între 3 profile (Dev, Prod, Test) folosind 3 flag-uri custom (DEVELOPMENT_SECURITY, RESET_PERSISTENT, TEST_MODE), fără modificarea codului sursă.
- Mapare CLI și Protecție la Eroare:
Scriptul `./flash_devices.sh` traduce argumentele CLI în opțiuni Kconfig și execută verificări de siguranță (avertisment automat la apelarea `--prod` fără `--persist`).
- Execuție Optimizată (Build Secvențial + Flash Paralel):
Compilare ordonată pentru stabilitatea mediului west, urmată de scriere simultană pe ambele plăci utilizând ID-urile unice J-Link.
- Depozit Unic de Configurare și Stare Curată:
Decuplare totală a parametrilor hardware prin `config.env` și suport pentru ștergerea integrală NVM prin `--factory-reset`.

```
george@george-KLVL-WXX9:~/Git-Projects/ZigBee-Gateway_nRF52840-DK$ ./flash_devices.sh -h

=====
Mode: DEVELOPMENT (auto-join with static key)
Storage: VOLATILE (resets NVRAM on boot)
Flash action: NORMAL
=====

Usage: ./flash_devices.sh {switch|coord|all} [--dev|--prod] [--persist] [--factory-reset]

Targets:
switch      Flash the End Device (BMP180 sensor)
coord       Flash the Network Coordinator
all         Build and flash both devices

Modes:
--dev       Development mode: auto-join with static key (default)
--prod      Production mode: manual UART pairing required

Options:
--persist   Enable NVRAM persistence (do not erase network config on boot)
--factory-reset Force flash erase (clean NVRAM settings before flashing)

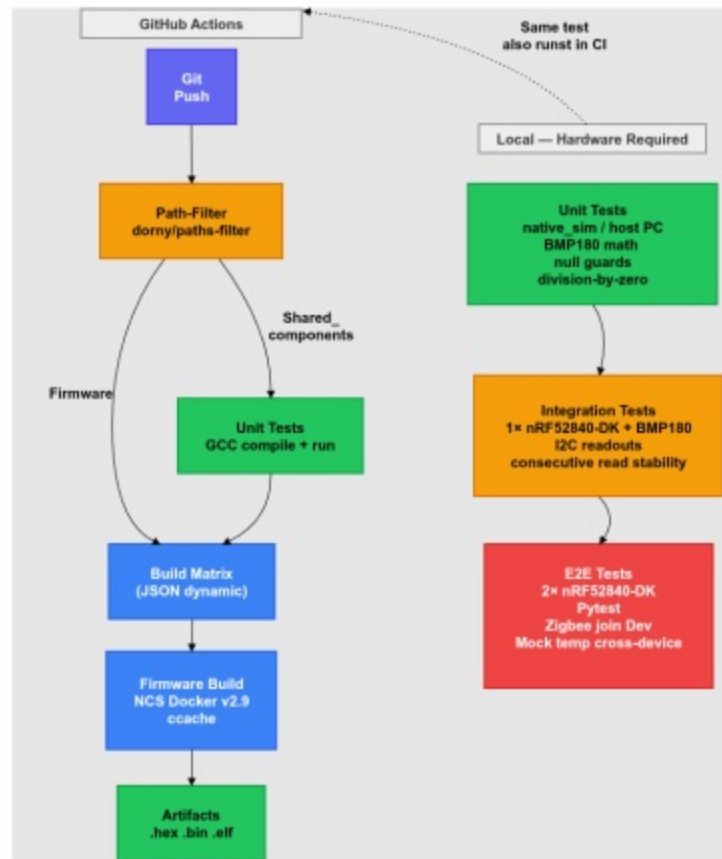
Examples:
./flash_devices.sh all           # dev mode (default)
./flash_devices.sh all --prod    # production mode (volatile)
./flash_devices.sh all --prod --persist # production mode (persistent)
./flash_devices.sh all --prod --persist --factory-reset # clean settings, then persistent
```

Rolul fiecărui nivel

Unit — verifică logica în izolare, fără hardware, rulează și în CI

Integration — verifică că driverul funcționează pe placa reală

E2E — verifică că cele 2 plăci comunică corect prin Zigbee

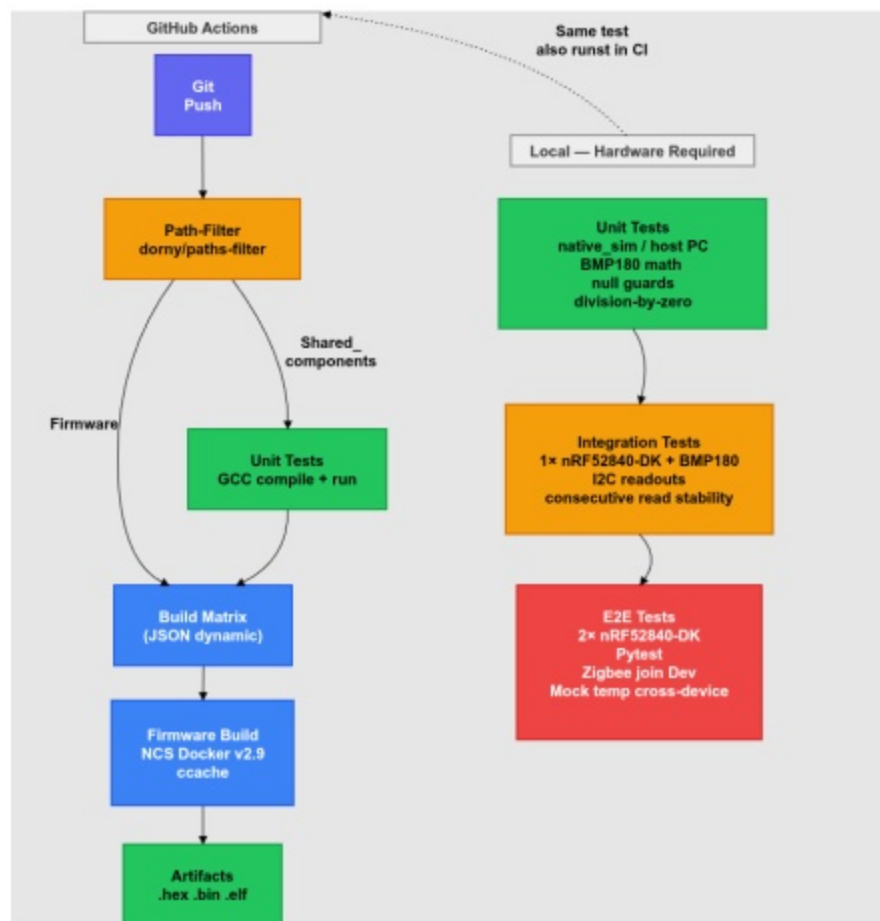


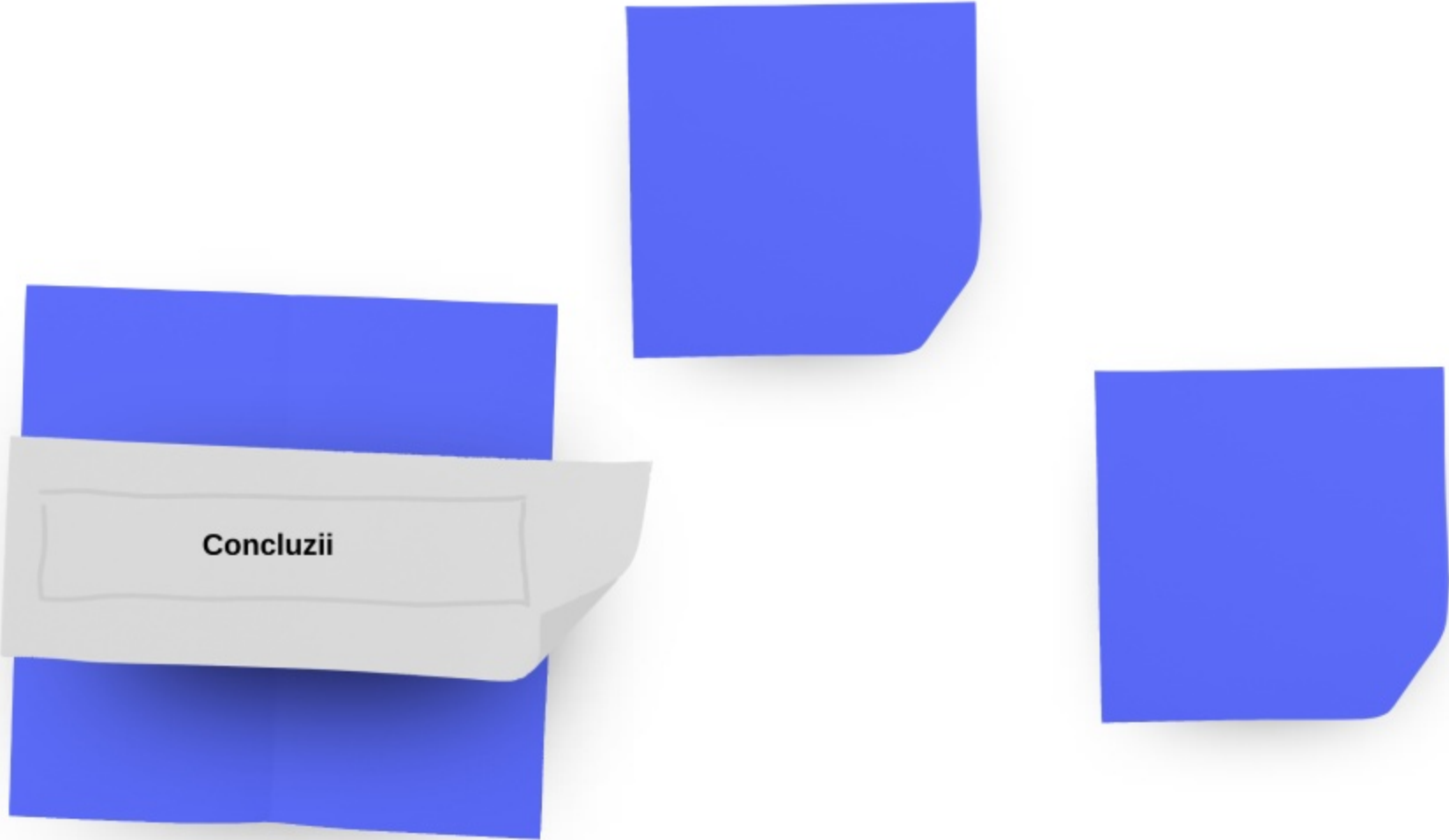
rui nivel

olare, fără hardware,

riverul funcționează pe

olăci comunică corect prin





Concluzii

